

FIRE DETECTION SYSTEM

Mount Agent API Guide

Local Control Interface and Remote Device Protocol
JSON commands, telemetry, security, and Python examples

Protocol: 1.0

Edition: Developer Guide

Updated: August 6, 2026

1. Overview

The Mount Agent starts with `fire-detector.service`. Both API surfaces route commands through the existing ODriveMonitor safety layer, so configured motion limits, drive availability, and enable conditions are enforced consistently.

Interface	Transport	Purpose	Default
Local Control Interface	Unix socket <code>/run/fire-detector/mount-agent.sock</code>	Low-latency control by applications running on the Mount computer.	Enabled
Remote Device Protocol	Outbound TCP/TLS connection to a Socket Receiver	Central control and telemetry without exposing an inbound port on the Mount.	Disabled until credentials are configured

Wire format: Both interfaces use protocol 1.0 and newline-delimited JSON (NDJSON). Each message must be no larger than 64 KiB.

Transport and latency

- On-device clients should keep one persistent Unix domain socket open. It is the lowest-overhead control path and exposes no network port.
- For remote operation, the Mount keeps one persistent outbound TCP/TLS connection to the Receiver, avoiding a handshake per command and working behind routers/CGNAT.
- Local subscriptions support 100–60,000 ms intervals (up to 10 Hz). Remote telemetry supports a minimum `telemetry_sec` of 0.1 seconds.
- The dashboard WebSocket at `/ws/status` is a read-only status stream, not a Mount Agent control transport.

TCP is a byte stream: clients must buffer until a newline and must not assume that one `recv()` call contains exactly one JSON message. Keep the same connection for the complete lease/control session.

2. Message Structure

Command

```
{
  "version": "1.0",
  "type": "command",
  "message_id": "cmd-0001",
  "action": "mount.get_status",
  "params": {}
}
```

Field	Required	Description
-------	----------	-------------

version	Yes	Must be 1.0.
type	Yes	Must be command.
message_id	Yes	Unique ID used for correlation and duplicate detection.
action	Yes	Command name.
params	Yes	JSON object; use {} when empty.
control_lease_id	For mutating commands	ID returned by control.acquire.
expires_at	Recommended remotely	ISO 8601 timestamp with timezone.

Successful response

```
{
  "version": "1.0",
  "type": "response",
  "message_id": "rsp-...",
  "correlation_id": "cmd-0001",
  "action": "mount.get_status",
  "timestamp": "2026-07-24T02:10:00.000Z",
  "ok": true,
  "result": {
    "connected": true,
    "axes": [...]
  }
}
```

Error response

```
{
  "version": "1.0",
  "type": "response",
  "correlation_id": "cmd-0002",
  "action": "mount.goto",
  "ok": false,
  "error": {
    "code": "CONTROL_LEASE_REQUIRED",
    "message": "Acquire and maintain a control lease before sending this command.",
    "details": {}
  }
}
```

3. Control Lease and Fail-Safe Behavior

Only one client may own control at a time. Acquire a lease, attach its ID to control commands, renew it before expiry, and release it when finished.

```
{
  "version": "1.0",
  "type": "command",
  "message_id": "lease-1",
  "action": "control.acquire",
  "params": {
    "lease_ms": 5000
  }
}
```

- Lease range: 500–30,000 ms.
- The session lease is released when its connection closes.
- `mount.stop`, `mount.disable`, `axis.stop`, and `axis.disable` do not require a lease.
- Non-zero velocity commands require `command_timeout_ms` from 100–5,000 ms. The watchdog commands zero velocity when updates stop.

4. Command Reference

System and control

Action	Lease	Purpose / parameters
system.hello	No	Protocol, device type, and capabilities.
system.ping	No	Agent liveness and timestamp.
system.get_info	No	System and device information.
system.get_health	No	Current system/drive health snapshot.
system.get_capabilities	No	Supported actions.
control.acquire	No	lease_ms
control.renew	No	lease_id
control.release	No	lease_id

Run-mode status

Every snapshot returned by `mount.get_status`, `system.get_health`, REST GET `/api/status`, and WebSocket `/ws/status` always includes:

Field	Type	Meaning
run_mode	string	simulation for the virtual model or hardware for real Drive/Encoder input.
simulation_mode	boolean	true only in Simulation Mode; otherwise false.

To query only the mode without the complete telemetry snapshot, call GET `/api/system/run-mode`:

```
{"ok":true,"run_mode":"simulation","simulation_mode":true}
```

Important: Clients must inspect `run_mode` before treating values as physical measurements. `connected: true` is also reported by Simulation Mode so the complete API workflow can be tested.

Mount

Action	Lease	Parameters
mount.get_status	No	None; returns the complete snapshot.
mount.enable	Yes	None.
mount.disable	No	None.
mount.stop	No	None.
mount.goto	Yes	azimuth_deg and/or altitude_deg; optional max_velocity_deg_per_sec.

mount.set_velocity	Yes	azimuth_deg_per_sec and/or altitude_deg_per_sec; command_timeout_ms for non-zero values.
--------------------	-----	--

```
{
  "version": "1.0",
  "type": "command",
  "message_id": "goto-1",
  "action": "mount.goto",
  "control_lease_id": "LEASE_ID",
  "expires_at": "2026-07-24T02:10:05.000Z",
  "params": {
    "azimuth_deg": 30,
    "altitude_deg": 15,
    "max_velocity_deg_per_sec": 10
  }
}
```

Axis

Action	Lease	Parameters
axis.enable	Yes	axis
axis.disable	No	axis
axis.stop	No	axis
axis.goto	Yes	axis, position_deg, optional max_velocity_deg_per_sec.
axis.set_velocity	Yes	axis, velocity_deg_per_sec, command_timeout_ms.

Axis values: azimuth/azm and altitude/alt.

```
{
  "version": "1.0",
  "type": "command",
  "message_id": "axis-1",
  "action": "axis.goto",
  "control_lease_id": "LEASE_ID",
  "params": {
    "axis": "altitude",
    "position_deg": 20,
    "max_velocity_deg_per_sec": 5
  }
}
```

Pointing, calibration, and tuning

Action	Parameters	Notes
pointing.goto_altaz	Same as mount.goto	Lease required.
calibration.start	axis, minimum_position_deg, maximum_position_deg, current_amp, velocity_deg_per_sec	Lease required.
calibration.stop	axis	Stops calibration.
calibration.get_status	Optional axis	Read-only.
tuning.apply	axis, values	Temporary values; lease required.
tuning.save	axis	Writes drive configuration; lease required.

Warning: Calibration and tuning can materially change drive behavior. The default Remote allowlist excludes these commands.

5. Telemetry

```
{
  "version": "1.0",
  "type": "command",
  "message_id": "sub-1",
  "action": "subscribe",
  "params": {
    "topics": ["axis.telemetry", "sensor.status", "fault"],
    "interval_ms": 500
  }
}
```

Topic	Data
mount.status	Connection, health, timestamp, and axes.
axis.telemetry	Position, velocity, current, state, and errors.
sensor.status	CCW/CW limit sensor state.
drive.status	Connection, serials, bus power, and health.
fault	Drive error events.
calibration.status	Motor calibration progress.

6. Python Example 1 — Status and Telemetry

Complete source: `API-Doc/examples/01_local_status_and_telemetry.py`. It is read-only and safe to run while the service is active.

```
with socket.socket(socket.AF_UNIX, socket.SOCK_STREAM) as client:
    client.connect("/run/fire-detector/mount-agent.sock")
    stream = client.makefile("rwb")
    send(stream, command("mount.get_status"))
    print(receive(stream))
    send(stream, command("subscribe", {
        "topics": ["axis.telemetry", "sensor.status"],
        "interval_ms": 500
    }))
    print(receive(stream))      # subscription response
    for _ in range(10):
        print(receive(stream))  # telemetry events
```

```
cd /home/pi/Public/Fire-Detector
.venv/bin/python API-Doc/examples/01_local_status_and_telemetry.py
```

7. Python Example 2 — Safe Axis Control

Complete source: `API-Doc/examples/02_safe_axis_control.py`. Motion is locked by default with `ALLOW_MOTION = False`. The example reads status, acquires a lease, enables Altitude, performs a low-speed Goto, and always executes stop/disable in finally.

```

lease = mount.call("control.acquire", {"lease_ms": 5000})
lease_id = lease["result"]["lease_id"]
try:
    mount.call("axis.enable", {"axis": "altitude"}, lease_id)
    mount.call("axis.goto", {
        "axis": "altitude",
        "position_deg": 10.0,
        "max_velocity_deg_per_sec": 2.0
    }, lease_id)
finally:
    mount.call("axis.stop", {"axis": "altitude"})
    mount.call("axis.disable", {"axis": "altitude"})
    mount.call("control.release", {"lease_id": lease_id})

```

Before enabling motion: clear the working area, station an operator at the Mount, verify configured limits and direction, start with low current/speed, and provide an independent emergency stop.

8. Remote Device Protocol and Security

1. The Mount opens an outbound TCP connection.
2. TLS validates the CA and hostname; TLS 1.2 is the minimum.
3. The Mount sends Device ID and capabilities.
4. The receiver sends a nonce challenge.
5. The Mount responds with an HMAC-SHA256 signature over device ID, nonce, and timestamp.
6. After authentication, heartbeat, telemetry, and allowlisted commands begin.

```

"remote": {
  "enabled": true,
  "host": "receiver.example.internal",
  "port": 15001,
  "server_name": "receiver.example.internal",
  "device_id": "NARIT-001",
  "ca_file": "/etc/fire-detector/ca.pem",
  "client_cert_file": "/etc/fire-detector/device.crt",
  "client_key_file": "/etc/fire-detector/device.key",
  "secret_file": "/etc/fire-detector/device.secret",
  "heartbeat_sec": 15,
  "telemetry_sec": 1
}

```

The device secret must contain at least 32 bytes. Store it in a permission-restricted file or environment variable, never directly in AppSetting.JSON or source control.

9. Camera Frame Ingest API

The main system no longer opens the Thermal or Visible camera directly. An external camera module connects both cameras and sends already encoded JPEG/PNG frames to the HTTP API. The relay retains only the newest frame per camera, preventing queue latency at approximately 60 FPS.

Recommended at 60 FPS: binary WebSocket

Do not serialize image bytes as a JSON integer array and do not Base64-encode high-rate frames. Open one persistent connection to `ws://192.168.1.105:8000/ws/camera-stream?role=producer&source=live` and send one binary message per frame:

```
[uint32 big-endian JSON header length]
[UTF-8 JSON header]
[all remaining bytes: encoded JPEG/PNG]
```

```
{"camera":"thermal","mime_type":"image/jpeg",
  "sha256":"64-lowercase-hex-characters",
  "metadata":{"producer_sequence":1234,
    "captured_at":"2026-08-11T10:30:00.000Z",
    "width":640,"height":512}}
```

The image bytes remain outside JSON. WebSocket/TCP provides ordered reliable delivery, `producer_sequence` identifies source-side gaps, and optional `sha256` verifies application payload integrity.

Important: Disable WebSocket per-message compression (`compression=None` in Python). JPEG/PNG is already compressed; recompressing every frame wastes CPU and can substantially reduce frame rate.

```
.venv/bin/python API-Doc/examples/04_publish_camera_binary_ws.py \
  --camera thermal --frame thermal-frame.jpg --fps 60
```

HTTP compatibility API

Camera	Endpoint
Thermal	POST <code>/api/camera-stream/frame/thermal?source=live</code>
Visible	POST <code>/api/camera-stream/frame/visible?source=live</code>

curl example

```
curl -X POST "http://192.168.1.105:8000/api/camera-stream/frame/thermal?
source=live" \
  -H "Content-Type: image/jpeg" \
  -H 'X-Camera-Metadata: {"captured_at":"2026-08-
11T10:30:00.000Z","width":640,"height":512,"producer_sequence":1}' \
  --data-binary @thermal-frame.jpg
```


Successful response

```
HTTP/1.1 202 Accepted
Content-Type: application/json

{"ok":true,"camera":"thermal","source":"live","sequence":1234}
```

Continuous HTTP example

```
.venv/bin/python API-Doc/examples/03_publish_camera_frames.py \
--host 192.168.1.105 --port 8000 \
--camera thermal --frame thermal-frame.jpg --fps 60
```

- Content-Type must be image/jpeg OR image/png.
- Optional X-Camera-Metadata must be a JSON object.
- Each frame is limited to 16 MiB. Raw thermal sensor arrays are not accepted.
- Run a second process with --camera visible for the Visible feed.
- Check GET /api/camera-stream and view the MJPEG paths under /api/camera-mjpeg/live/...

Production guidance: Publish pre-encoded frames over a stable LAN, inspect every HTTP status, and reconnect or retry with backoff when the receiver is unavailable.

10. Errors and Client Guidance

Error code	Client action
UNSUPPORTED_VERSION	Use protocol 1.0.
INVALID_JSON, INVALID_MESSAGE_TYPE	Correct framing/schema.
MISSING_MESSAGE_ID	Generate a unique ID for every command.
MESSAGE_TOO_LARGE	Keep each message below 64 KiB.
RATE_LIMITED	Remain below 20 commands/sec and back off.
COMMAND_EXPIRED	Re-read state before creating a new command.
CONTROL_LEASE_BUSY	Do not take control from the current owner.
CONTROL_LEASE_REQUIRED	Acquire a lease on the same connection.
ACTION_NOT_ALLOWED	Action is outside the Remote allowlist.
INVALID_PARAMETER	Check values and configured safety limits.
COMMAND_FAILED	Read health/status; do not blindly retry motion.

Implementation references: src/fire_detector/mount_agent.py, src/fire_detector/web_server.py, and scripts/mount_agent_client.py.

Verification status — August 4, 2026

Automated integration tests cover a real Unix socket round trip, response correlation, duplicate protection, control leases, and telemetry subscription without commanding hardware. Run `PYTHONPATH=src .venv/bin/python -m unittest discover -s tests -v`. Production acceptance still requires a supervised hardware test of axis direction, software/physical limits, emergency stop, velocity watchdog, and Receiver reconnection.